

EXPERIMENT-03

C PROGRAMMING – DEBUGGING TOOLS

1. Fundamentals of Errors and Debugging

Key Definitions

- **Error:** A mistake or blunder made by the programmer in the source code.
- **Bug:** A flaw or fault in a computer program that cause it to produce an incorrect or unexpected result, or to behave in unintended ways.
- **Debugging:** The systematic process of finding, analysing, and removing bugs or detects in a computer program.

2. Classification of Errors in C

Error Type	Description	Primary cause	Example
Syntax Errors	Code violates the grammatical rules of the C Language.	Missing semicolons, unmatched braces, types in keywords.	int a = 10(semicolon missing).
Compilation/Type Error	Operation performed on incompatible types or missing declarations.	Assigning float string to hit int pointer, using undeclared variable.	int *p = "hello";
Linker Errors	The compiler generates object files but the linker cannot combine them.	Function declared but not defined, missing libraries.	Undefined reference to 'main'.
Runtime Errors	The program crashes or terminates abnormally during execution.	Division by zero accessing invalid memory addresses.	int x = 10/0.

Logical error	Code runs to completion without crashing, but output is incorrect.	Wrong mathematical formula, Incorrect loop condition.	for(i=0; i<5;i++) instead of <=5.
----------------------	--	---	-----------------------------------

Why does a Program Compile Successfully but produce incorrect output?

A compiler's job is to verify that your code obeys the **Syntax** and **type rules** of the C language, and to convert that code into machine instructions. The compiler **cannot understand intent**,

- If you write `sum = a+b`; the statement is completely valid C syntax. Therefore, compilation succeeds, but the resulting logic produces an erroneous output at execution time.

Case 1: Syntax & Compilation Errors

when the compiler encounters invalid code, it prints diagnostics specifying the file name, line number, column, and error description.

```
#include <stdio.h>

int main() {

int x = 10;

printf("the value of x is %d", x)
```

```
return 0;
```

```
}
```

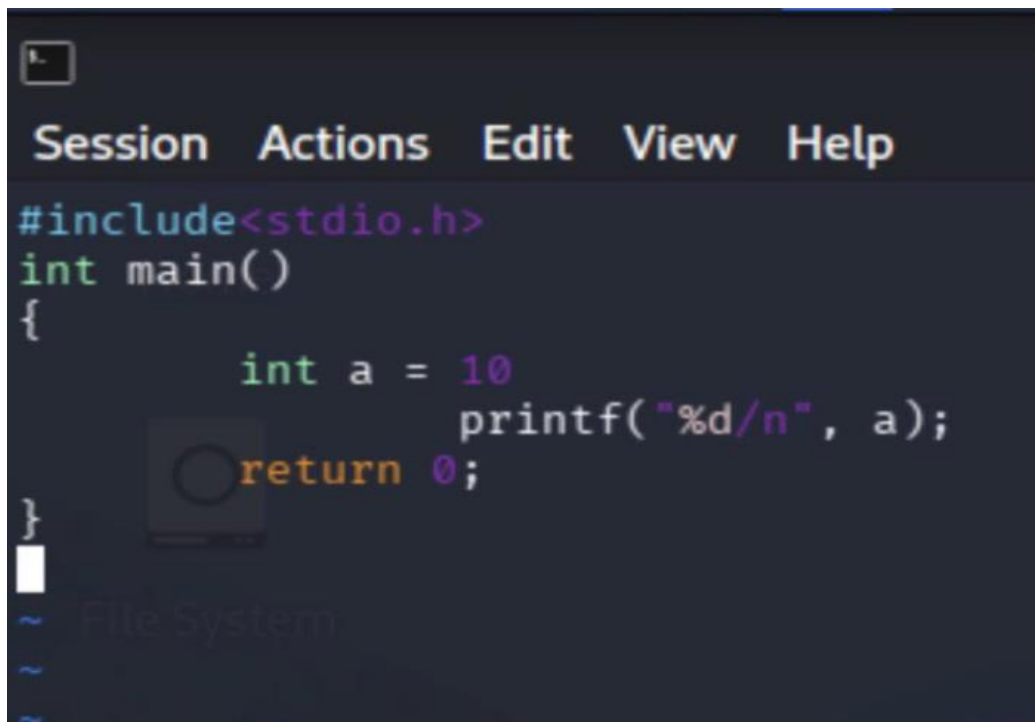
Compilation Command & Output

```
$ cc sample1.c
```

```
sample1.c:5:5:error:expected ',' or ';' before printf
```

Breakdown of Error Message

- **Filename:sample1.c**
- **Line&column:** Line 5, Column 5
- **Type of Message:** error
- **Description:**Expected, or ; before printf statement

A screenshot of a code editor with a dark theme. The editor has a menu bar at the top with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The code being edited is a C program:

```
#include<stdio.h>
int main()
{
    int a = 10
    printf("%d/n", a);
    return 0;
}
```

 The cursor is positioned at the end of the line containing 'return 0;'. A small, semi-transparent error icon is visible over the code. On the left side of the editor, there is a sidebar with a 'File System' view showing a tree structure with '~' symbols representing directories.

```
Session Actions Edit View Help

(kali㉿kali)-[~]
$ vi sample1.c

(kali㉿kali)-[~]
$ cc sample1.c
sample1.c: In function 'main':
sample1.c:5:17: error: expected ',', or ';' before 'printf'
    5 |         printf("%d/n", a);
      |         ~~~~~
(kali㉿kali)-[~]
$
```

This is an example of syntax and compilation error where er didn't place an semicolon after the `int a = 10` and the second pic depicts the compilation error where the compiler error detected the bug in the code.

```
Session Actions Edit View Help

#include<stdio.h>
int main()
{
    int a = 10;
    printf("%d/n", a);
    return 0;
}
```

```
Session Actions Edit View Help

(kali㉿kali)-[~]
$ cc sample1.c

(kali㉿kali)-[~]
$ vi sample1.c

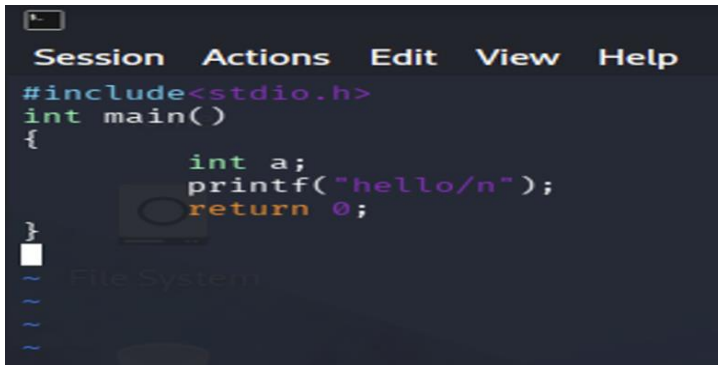
(kali㉿kali)-[~]
$ cc sample1.c

(kali㉿kali)-[~]
$ ./a.out
10/n
(kali㉿kali)-[~]
$
```

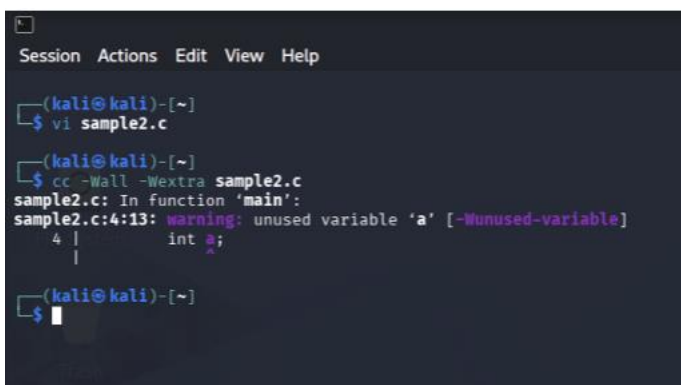
In the above two pics we have debugged the code and it is running how it is supposed to.

Case 2: Compiler Warnings (-Wall, -Wextra)

A warning indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do not stop compilation, but ignoring them frequently leads to bugs at runtime



```
Session Actions Edit View Help
#include<stdio.h>
int main()
{
    int a;
    printf("hello/n");
    return 0;
}
```



```
Session Actions Edit View Help
(kali@kali)-[~]
$ vi sample2.c
(kali@kali)-[~]
$ cc -Wall -Wextra sample2.c
sample2.c: In function 'main':
sample2.c:4:13: warning: unused variable 'a' [-Wunused-variable]
    4 |         int a;
      |         ^
(kali@kali)-[~]
$
```

This code will run if we compile it normally with the command `cc sample2.c` but if we use the command `cc-Wall-Wextra sample2.c` it will show an unused variable.

Case 3: printf-based debugging (Logical bug)

Before using complex debugging tools, simple program flow and state inspection can be performed using output statements(`printf`).

```
Session Actions Edit View Help
#include<stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i = 1; i < 5; i++)
    {
        printf("DEBUG: i = %d/n", i);
        sum = sum + i;
        printf("DEBUG: sum = %d/n", sum);
    }
    return 0;
}
```

```
Session Actions Edit View Help
kali@kali:~$ vi sample5.c
kali@kali:~$ cc sample5.c
kali@kali:~$ ./a.out
DEBUG: i = 1/nDEBUG: sum = 1/nDEBUG: i = 2/nDEBUG: sum = 3/nDEBUG: i = 3/nDEBUG: sum = 6/nDEBUG: i = 4/nDEBUG: sum = 10/n
kali@kali:~$
```

In this code there's a logical error the code runs fine but we need an answer for the sum of first five numbers i.e 1-5 but the code gives us answer to 1-4 to fix it we will just add $i \leq 5$ in the source code

```
Session Actions Edit View Help
#include<stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i = 1; i ≤ 5; i++)
    {
        printf("DEBUG: i = %d/n", i);
        sum = sum + i;
        printf("DEBUG: sum = %d/n", sum);
    }
    return 0;
}
```

```
Session Actions Edit View Help
~(kali@kali)~[-]
└─$ v: samples.c
~(kali@kali)~[-]
└─$ cc samples.c
~(kali@kali)~[-]
└─$ ./a.out
DEBUG: i = 1/DEBUG: sum = 1/DEBUG: i = 2/DEBUG: sum = 3/DEBUG: i = 3/DEBUG: sum = 6/DEBUG: i = 4/DEBUG: sum = 10/
~(kali@kali)~[-]
└─$ v: samples.c
~(kali@kali)~[-]
└─$ cc samples.c
~(kali@kali)~[-]
└─$ ./a.out
DEBUG: i = 1/DEBUG: sum = 1/DEBUG: i = 2/DEBUG: sum = 3/DEBUG: i = 3/DEBUG: sum = 6/DEBUG: i = 4/DEBUG: sum = 10/DEBUG: i = 5/DEBUG: sum = 15/
~(kali@kali)~[-]
└─$ v: samples.c
~(kali@kali)~[-]
└─$
```

Now we will use GDB to Debug the code

Case 4: Debugging a logical error using GDB

```
1 #include<stdio.h>
2 int main()
3 {
4     int i, sum = 0
5     for(i = 1; i < 5; i++)
6     {
7         sum = sum + i;
8     }
9     printf("sum = %d/n", sum);
10    return 0;
11 }
12 |
```

```

Session Actions Edit View Help
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type 'help'.
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample...
(gdb) list
1      #include<stdio.h>
2      int main()
3      {
4          int i, sum = 0;
5          for(i = 1; i <= 5; i++)
6          {
7              sum = sum + i;
8          }
9          printf("sum = %d\n", sum);
10         return 0;
(gdb) break 7
Breakpoint 1 at 0x1151: file sample.c, line 7.
(gdb) run
Starting program: /home/kali/sample
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample.c:7
7      sum = sum + i;
(gdb) print i
$i = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7      sum = sum + i;
1: i = 2
2: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7      sum = sum + i;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7      sum = sum + i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
sum = 10/n[Inferior 1 (process 25440) exited normally]
(gdb)

```

In this code this logical error is same as the case-3 but in this we debugged with the help of GDB and we wrote the code in gedit instead VI,

If we fix the source code it will be like this

```
1 #include<stdio.h>
2 int main()
3 {
4     int i, sum = 0;
5     for(i = 1; i ≤ 5; i++)
6     {
7         sum = sum + i;
8     }
9     printf("sum = %d/n", sum);
10    return 0;
11 }
12
```

```

Session Actions Edit View Help
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample...
(gdb) list
1      #include<stdio.h>
2      int main()
3      {
4          int i, sum = 0;
5          for(i = 1; i <= 5; i++)
6          {
7              sum = sum + i;
8          }
9          printf("sum = %d/n", sum);
10         return 0;
(gdb) break 7
Breakpoint 1 at 0x1151: file sample.c, line 7.
(gdb) run
Starting program: /home/kali/sample
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample.c:7
7      sum = sum + i;
(gdb) print i
$1 = 1
(gdb) display sum
1: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7      sum = sum + i;
1: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7      sum = sum + i;
1: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7      sum = sum + i;
1: sum = 6
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7      sum = sum + i;
1: sum = 10
(gdb) continue
Continuing.
sum = 15/n[Inferior 1 (process 29586) exited normally]
(gdb) █

```

Now the answer is correct

Case 5: Debugging functions (next vs step)

In this we will using two steps

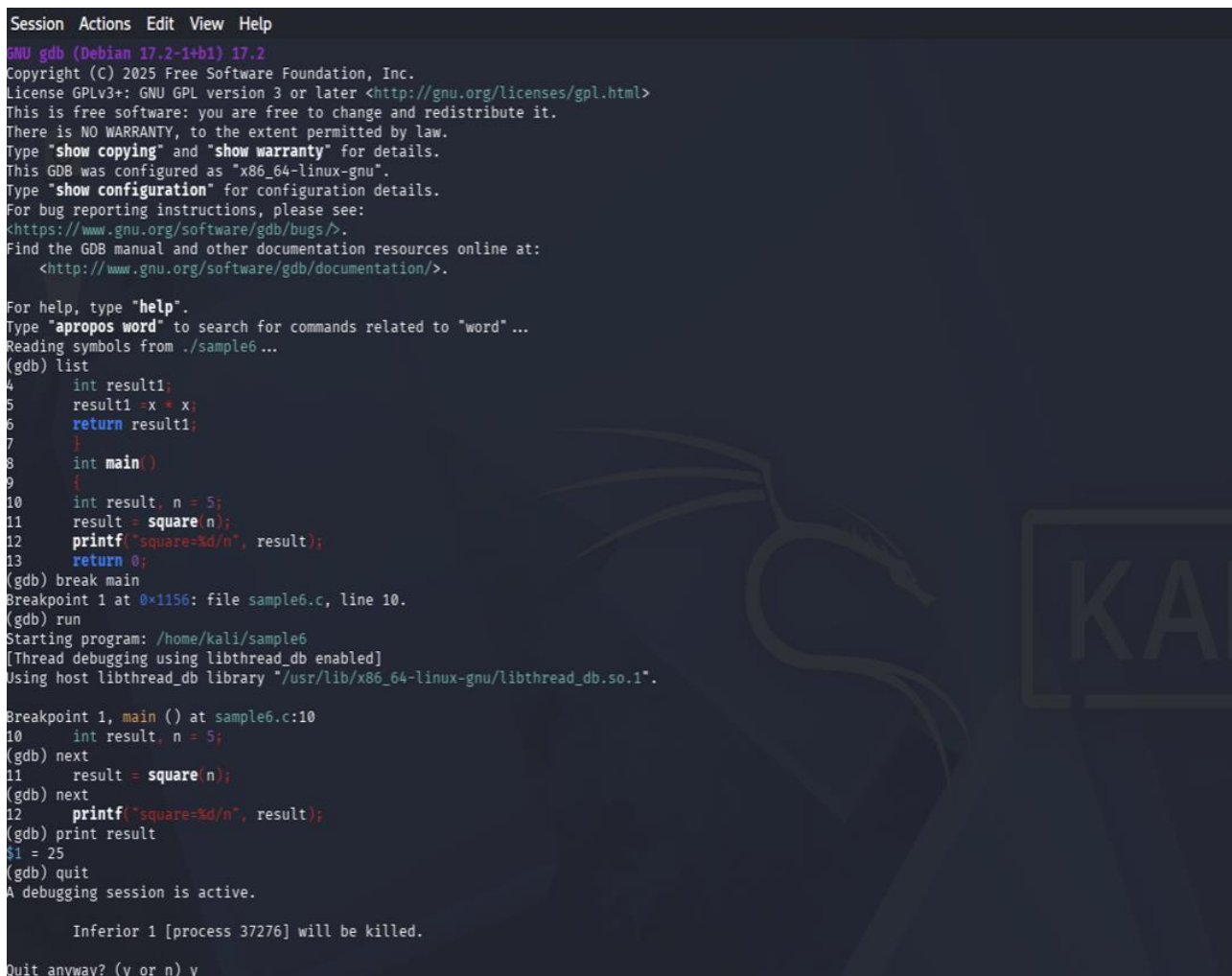
1) next

2)step

By using next to debug



```
sample6.c
1 #include <stdio.h>
2 int square(int x)
3 {
4     int result1;
5     result1 = x * x;
6     return result1;
7 }
8 int main()
9 {
10    int result, n = 5;
11    result = square(n);
12    printf("square=%d/n", result);
13    return 0;
14 }
```



```
Session Actions Edit View Help
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample6...
(gdb) list
4      int result1;
5      result1 = x * x;
6      return result1;
7  }
8  int main()
9  {
10     int result, n = 5;
11     result = square(n);
12     printf("square=%d/n", result);
13     return 0;
(gdb) break main
Breakpoint 1 at 0x1156: file sample6.c, line 10.
(gdb) run
Starting program: /home/kali/sample6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample6.c:10
10     int result, n = 5;
(gdb) next
11     result = square(n);
(gdb) next
12     printf("square=%d/n", result);
(gdb) print result
$1 = 25
(gdb) quit
A debugging session is active.

    Inferior 1 [process 37276] will be killed.

Quit anyway? (y or n) y
```

Now we will use step to debug

```
(kali@kali)~[~]
$ gedit sample6.c

** (gedit:39110): WARNING **: 13:18:45.529: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:39110): WARNING **: 13:18:45.529: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)~[~]
$ gdb ./sample6
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample6 ...
(gdb) break main
Breakpoint 1 at 0x1156: file sample6.c, line 10.
(gdb) run
Starting program: /home/kali/sample6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample6.c:10
10     int result, n = 5;
(gdb) next
11     result = square(n);
(gdb) step
square (x=5) at sample6.c:5
5     result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6     return result1;
(gdb) print result
X No symbol "result" in current context.
(gdb) print result1
$2 = 25
(gdb) |
```

Case 6: Debugging runtime errors using backtrace(bactrace/bt)

```
sample/.c
~
1 #include <stdio.h>
2 int divide(int a, int b)
3 {
4     return a / b;
5 }
6 int calculate(int x)
7 {
8     return divide(x, 0);
9 }
10 int main()
11 {
12     int result = calculate(10);
13     printf("result:%d/n", result);
14     return 0;
15 }
```

```

Session Actions Edit View Help
└─$ cc -g sample7.c -o sample7

(kali@kali)-[~]
└─$ gdb ./sample7
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample7 ...
(gdb) run
Starting program: /home/kali/sample7
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x0000555555555147 in divide (a=10, b=0) at sample7.c:4
4       return a / b;
(gdb) backtrace
#0 0x0000555555555147 in divide (a=10, b=0) at sample7.c:4
#1 0x0000555555555166 in calculate (x=10) at sample7.c:8
#2 0x000055555555517a in main () at sample7.c:12
(gdb) frame 0
#0 0x0000555555555147 in divide (a=10, b=0) at sample7.c:4
4       return a / b;
(gdb) print b
$1 = 0
(gdb) frame 1
#1 0x0000555555555166 in calculate (x=10) at sample7.c:8
8       return divide x, 0;
(gdb) print x
$2 = 10
(gdb) quit

```

Case 7: Debugging array out-of-bounds error GDB

```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
└─$ gedit sample8.c

** (gedit:59967): WARNING **: 13:56:32.607: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:59967): WARNING **: 13:56:32.607: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)-[~]
└─$ cc -g sample8.c -o sample8

(kali@kali)-[~]
└─$ gdb ./sample8
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample8 ...
(gdb) break 8
Breakpoint 1 at 0x114a: file sample8.c, line 8.
(gdb) run
Starting program: /home/kali/sample8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample8.c:8
8       scanf("%d/n", &a[i]);
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) n
X Program not restarted.
(gdb) print &a[0]
$1 = (int *) 0x7fffffffd20
(gdb) print &a[4]
$2 = (int *) 0x7fffffffd30
(gdb) print &a[5]
$3 = (int *) 0x7fffffffd34
(gdb) █

```

Case 8: Segmentation faults (SIGSEGV) using GDB

```
Open sample9.c
#include <stdio.h>
int main()
{
int *p = NULL;
*p = 10;
printf("%d/n", *p);
return 0;
}
```

```
(kali@kali)-[~]
$ gdb ./sample9
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample9...
(gdb) run
Starting program: /home/kali/sample9
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at sample9.c:5
warning: Source file is more recent than executable.
5      int *p = 0x;
(gdb) print p
$1 = (int *) 0x0
(gdb) prit *p
X Undefined command: "prit". Try "help".
(gdb) print *p
X Cannot access memory at address 0x0
(gdb) █
```

This before we wrote the source code correctly

```
1 #include <stdio.h>
2 int main()
3 {
4     int x;
5     int *p = &x;
6     *p = 10;
7     printf("%d/n", *P);
8     return 0;
9 }
10
```

```
Session Actions Edit View Help
** (gedit:64542): WARNING **: 14:05:05.666: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:64542): WARNING **: 14:05:05.666: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)~$ cc -g sample9.c -o sample9

(kali@kali)~$ gdb ./sample9
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample9...
(gdb) run
Starting program: /home/kali/sample9
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at sample9.c:5
5      *p = 10;
(gdb) print p
$1 = (int *) 0x0
```

In this experiment we learned how to debug source code via multiple methods like GDB by printf and many more